AIMERZ.ai
Aim.Act.Achieve

NOTES

ENums - I

What are ENums
Advantages of ENums
Defining and Syntax of ENum

What are Enums in Java?

Enums (short for enumerations) in Java are a special type of class that represents a group of constants (unchangeable values). Each constant in an enum is by default public, static, and final, meaning they are constants and can be accessed without creating an instance of the enum.

Enums are often used to represent a fixed set of known values such as days of the week, directions (NORTH, SOUTH, EAST, WEST), states in a finite state machine, or months of the year.
In Java, enums are a type-safe alternative to traditional integer constants. They help prevent errors, such as using invalid or unexpected values, making your code more readable and less prone to bugs.

Example of enum usage:

```Java
enum Day {
      SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}
public class EnumExample {
      public static void main(String[] args) {
      Day today = Day.MONDAY;
      if (today == Day.MONDAY) {
            System.out.println("It's Monday! Time to work.");
      }
      }
}
```

Advantages of Enums in Java

Type Safety:
Enums are type-safe, which means only valid enum values can be assigned to enum variables. You can't accidentally assign a non-enum value (like a string or integer) to an enum variable.

```Java
Day today = Day.MONDAY;  // Correct

Day today = 1;    // Compilation error
```

Readability and Maintainability:
Enums make your code more readable by replacing magic numbers or strings with meaningful names, improving maintainability. For example, using MONDAY instead of 1 to represent a day is clearer.

Better Control Over Constants:
Enums can contain fields, methods, and constructors, giving you more control than using traditional constants. You can define behaviors that are specific to enum values, making them more versatile.

Single Instance:
Enums provide a single instance of each enum constant. They are internally handled as static final instances.

Use in Switch Statements:
Enums can be used in switch statements, enhancing code clarity. This wasn't possible with earlier Java versions using string or integer constants.

```java
switch (today) {
      case MONDAY:
      System.out.println("Start of the week.");
      break;
      case FRIDAY:
      System.out.println("Almost the weekend!");
      break;
      default:
      System.out.println("Another regular day.");
}
```

Defining and Syntax of Enum

In Java, enums are defined using the enum keyword. Enums can contain methods, fields, constructors, and implement interfaces, just like a regular class.

Basic Enum Syntax:

```java
enum EnumName {
    CONSTANT1, CONSTANT2, CONSTANT3
}
enum TrafficSignal {
    RED, GREEN, YELLOW
}
```

Enum with fields and methods:

```java
enum TrafficSignal {
    RED("Stop"), GREEN("Go"), YELLOW("Slow down");

    private String action;

    // Constructor
    private TrafficSignal(String action) {
        this.action = action;
    }

    // Getter method
    public String getAction() {
        return action;
    }
}

public class EnumWithFieldExample {
    public static void main(String[] args) {
        TrafficSignal signal = TrafficSignal.RED;
        System.out.println(signal + ": " + signal.getAction());  //
Output: RED: Stop
    }
}
```

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan